

CO3

1. Explain the key stages of a CI/CD pipeline and analyze how each stage contributes to automated software delivery.
2. Implement a Git workflow for feature development using branching and pull requests, and explain each step involved.
3. Compare Git Flow, GitHub Flow, and Trunk-Based Development strategies with suitable examples.
4. Develop a GitHub Actions workflow triggered by a push event and explain how jobs, steps, and runners automate CI/CD.
5. Analyze the importance of the Software Testing Pyramid and how it improves software quality and efficiency.
6. Compare the unit build stage and container build stage with examples, and explain their role in DevOps workflows.
7. Implement Docker layer caching techniques and explain how Dockerfile instruction ordering improves build performance.
8. Analyze different cache optimization strategies in Docker builds and their impact on CI/CD efficiency.
9. Explain the process of generating an SBOM using tools like Syft and analyzing vulnerabilities using Gype.
10. Examine the role of vulnerability scanning and SBOM generation in ensuring container security and reliability.

CO4

1. Explain the role of IAM and RBAC in Kubernetes and how they help secure cluster resources.
2. Demonstrate how Kubernetes Secrets are created, managed, and used securely within applications.
3. Compare Kubernetes Secrets and Sealed Secrets in terms of security, usability, and GitOps compatibility.
4. Apply rollout and rollback strategies in Kubernetes and analyze their importance in minimizing deployment risks.
5. Explain the concept of admission controllers and their role in enforcing security and governance policies.
6. Analyze the use of observability tools such as logs and node metrics in troubleshooting Kubernetes clusters.
7. Examine common Kubernetes failure scenarios (e.g., CrashLoopBackOff) and explain how they can be diagnosed.

8. Apply RBAC principles using roles, role bindings, and service accounts to enforce least-privilege access.
9. Analyze cost optimization techniques in Kubernetes clusters and their impact on performance and efficiency.
10. Analyze how Architecture Decision Records (ADRs) improve maintainability, traceability, and decision-making in Kubernetes-based systems.